HUM1012 : Logic and Language Structure

(LT)

Slot : B21 (Mon 04.25 pm – 05.55 pm) + E14 (Tue 02.50 pm – 04.20 pm)
D21 (Tue 01.15 pm – 02.45 pm) B14 + (Wed 02.50 pm – 04.20 pm)

Venue : AB02 – 301 (B21+E14)
AB02 – 301 (B14+D21)

Prepared and Presented by

Dr. Velmani Ramasamy, M.E., Ph.D., SMIEEE., MACM.,
Associate Professor (Grade 2)
School of SCAI,
VIT Bhopal University,
Bhopal, Madhya Pradesh.

MODULE 2 Syntax and Semantics Interface in Natural Language 3

Context free grammar, POS tagging, constituency, trees, sentence, utterance, proposition, sense, reference, referring expressions, predicates, universe of discourse, deixis and definiteness, lexical semantics.

COURSE OUTCOMES:

CO1: Understanding approaches to syntax and semantics in NLP.

Syntax and Semantics Interface in Natural Language (NLP)

- Interface between syntax and semantics is crucial for understanding how the *structure* (syntax) of a sentence relates to its *meaning* (semantics).
- **Syntax**: Rules that govern the structure of sentences (e.g., word order, grammatical relations).
- **Semantics**: The meaning conveyed by those sentences (e.g., who did what to whom).

2.1 Context Free Grammer (CFG)

- CFG is a formal set of rules used to describe the syntactic structure of sentences in a language.
- CFG is defined by a set of production rules that describe how symbols in a language can be combined to form valid sentences.
- It helps machines understand how words can be combined to form grammatically correct phrases and sentences.
- It helps computers parse and understand the structure of sentences.

CFG consists of 4 tuples:

- ✓ **Terminals (Σ)** – Actual symbols/words in the language (e.g., "dog", "runs").
- ✓ **Non-terminals (V)** – Symbols representing grammatical categories (e.g., Sentence (S), Noun Phrase (NP), Verb Phrase (VP)).
- ✓ **Start Symbol (S)** – The symbol from which parsing begins (usually S for Sentence).
- ✓ **Production Rules (P)** – Rules that show how non-terminals can be expanded into other non-terminals or terminals.

Non Terminals:

S (Sentence) is a sentence that consists of a noun phrase followed by a verb phrase.

NP (Noun Phrase) consists of a determiner and a noun.

VP (Verb Phrase) consists of a verb and another noun phrase.

N (Noun) is a word that names a person, place, thing, or idea.

Examples: dog, cat, apple, teacher, school

V (Verb) is a word that expresses an action or a state of being.

Examples: run, eat, is, like, chased

Det (Determiner) is a word placed in front of a noun to specify quantity or definiteness.

Examples: a, an, the, some, this

Terminals:

Terminals are actual words: "the", "cat", "chased", "a", "dog".

“the cat chased a dog”

- CFG for a small subset of English:

S → NP VP

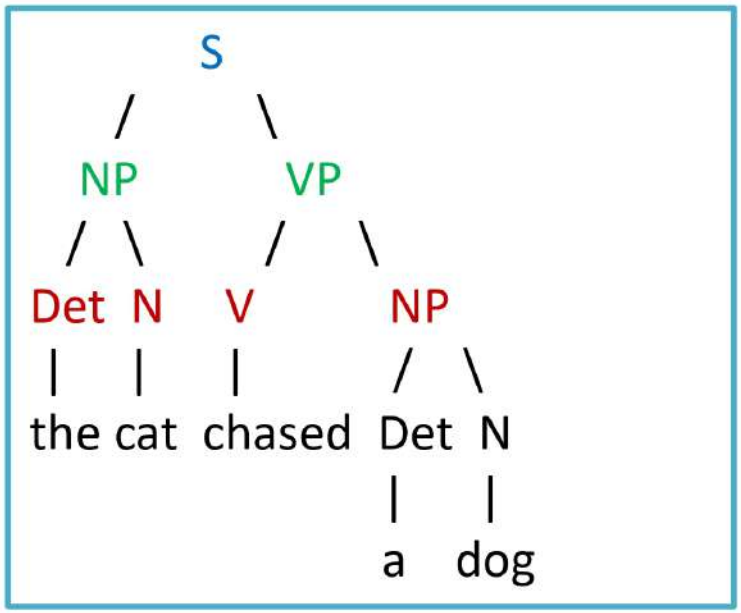
NP → Det N

VP → V NP

Det → "the" | "a"

N → "cat" | "dog"

V → "chased"



S

→ NP VP

→ Det N VP

→ "the" N VP

→ "the" "cat" VP

→ "the" "cat" V NP

→ "the" "cat" "chased" NP

→ "the" "cat" "chased" Det N

→ "the" "cat" "chased" "a" N

→ "the" "cat" "chased" "a" "dog"

How CFG is Useful in NLP?

- ✓ Syntax Analysis: Checks if a sentence is grammatically valid.
- ✓ Parsing: Helps machines break down sentences into structured parts.
- ✓ Understanding: Makes language understandable for chatbots, translators, etc.
- ✓ Machine Translation: Helps rearrange grammar in different languages.

- ✓ Helps parse and analyze sentence structure.
- ✓ Used in syntax checking.
- ✓ Helps machines understand grammar of a language.
- ✓ Useful in speech recognition, translation, Chatbots, etc.

Types of CFG

Type	Key Feature	Use Case
Regular CFG	Linear, like regular grammar	Simple pattern matching
Unambiguous CFG	One unique parse tree per sentence	Compiler design, structured parsing
Ambiguous CFG	Multiple parse trees possible	Natural language processing
Recursive CFG	Allows repeated/recursive structures	Generating long/nested sentences
Epsilon-Producing CFG	Allows empty string derivation	Optional parts of sentences
Lexical CFG	Token-level grammar	Tokenization, POS tagging
Syntactic CFG	Structure-level grammar	Syntax parsing in NLP
Probabilistic CFG (PCFG)	Grammar with probabilities	Statistical NLP

Type	Definition	Grammar Example	Generated Sentence(s)
Regular CFG	CFG with rules that generate only one non-terminal per side (linear)	S → aS | bS | ε	"a", "ab", "bba"
Strongly Regular CFG	CFGs that are strictly left-linear or right-linear and can be converted to FSM (Finite State Transducer)	S → aA, A → bS | ε	"ab", "abab", "ababb"
Unambiguous CFG	Every string has **only one** valid parse tree	S → aSb | ε	"ab", "aabb", "aaabbb"
Ambiguous CFG	Some strings have **multiple parse trees**	E → E + E | E * E | num	"num + num * num" (ambiguous parse)
Recursive CFG	Grammar that calls itself recursively to generate infinite patterns	S → aS | a	"a", "aa", "aaa", etc.
Epsilon-Producing CFG	Grammar that includes the empty string (ε) as a valid derivation	S → aS | ε	"a", "aa", "" (empty)
Lexical CFG	Defines rules at the **word/token level** (POS like N, V, Det)	N → "dog" | "cat“ V → "runs"	"dog", "cat" (nouns)
Syntactic CFG	Defines structure rules like phrases and sentences	S → NP VP; NP → Det N; VP → V NP	"the cat eats an apple"
Probabilistic CFG (PCFG)	CFG + probabilities to handle ambiguity	S → NP VP [0.9] S → VP NP [0.1]	Likely parse: "the dog chased the cat"

CFG Tools

Tool / Library	Domain	Description	Use Cases
NLTK (Python)	NLP / Education	Natural Language Toolkit — supports CFG definition and parsing	Parsing sentences, grammar teaching
ANTLR	Compiler Design	Powerful parser generator for reading, processing, executing, or translating structured text	Language processing, compilers
Lark (Python)	General Parsing	Modern parser for context-free grammars, supports Earley and LALR(1) parsers	Fast parsing, tree construction
Bison (GNU Bison)	Compiler Construction	A parser generator similar to Yacc that can handle CFGs (used with Lex)	Language and compiler development
Yacc / Lex	Compiler Design	Yet Another Compiler Compiler — traditional tools for CFG-based parsing	Parsing code, DSLs
spaCy (Python)	NLP	Not CFG-based, but uses statistical models and dependency parsing	Practical NLP tasks (POS, NER, parsing)
Stanford Parser	NLP	Java-based parser using probabilistic CFGs	Sentence parsing, syntactic tree generation
ParseKit (iOS/macOS)	App Development	Cocoa framework for parsing structured text based on CFGs	Mac/iOS grammar-based parsing
Irony (C#)	.NET Applications	.NET language implementation toolkit with CFG support	Designing DSLs, interpreters in C#
TextX (Python)	DSLs / Parsing	Builds parsers and meta-models for domain-specific languages	Language prototyping, code generation

2.2 POS (Part-of-Speech) Tagging

- POS (Part-of-Speech) Tagging in Natural Language Processing (NLP) is the process of assigning a part of speech to each word in a sentence based on its meaning and context.
- POS tagging in NLP are widespread, as POS tags help systems understand the grammatical structure and contextual meaning of words.
- Common parts of speech include noun, verb, adjective, adverb, etc.

Importance of POS tagging:

- ✓ Helps in syntactic parsing
- ✓ Aids information retrieval
- ✓ Improves machine translation
- ✓ Supports text-to-speech and question answering systems.

POS Tagging Techniques

Technique	Description
Rule-based	Uses hand-written linguistic rules
Statistical	Uses probabilities (e.g., HMM - Hidden Markov Model, CRF – Conditional Random Fields)
Machine Learning	Trained on labeled corpora (e.g., using SVM, Neural Nets)
Deep Learning	Uses RNNs, Bi-LSTM (Long Short Term Memory), Transformers

Common POS Tags

Tag	Part of Speech	Example
NN	Noun (Singular)	book, car
NNS	Noun (Plural)	books, cars
VB	Verb (Base Form)	run, eat
VBD	Verb (Past Tense)	ran, ate
JJ	Adjective	red, big
RB	Adverb	quickly, silently
PRP	Personal Pronoun	he, she, it
IN	Preposition	in, on, at
DT	Determiner	the, a, an

Penn Treebank PoS Tags

Tag	Meaning	Example
CC	Coordinating conjunction	and, but, or
CD	Cardinal number	one, two, 100
DT	Determiner	the, a, an
EX	Existential there	there (as in "there is")
FW	Foreign word	d'accord, ¡hola!
IN	Preposition/Subordinating conjunction	in, of, like, because
JJ	Adjective	big, red, quick
JJR	Adjective, comparative	bigger, faster
JJS	Adjective, superlative	biggest, fastest
LS	List item marker	1), A), •
MD	Modal verb	can, will, must
NN	Noun, singular	cat, computer
NNS	Noun, plural	cats, computers
NNP	Proper noun, singular	John, London
NNPS	Proper noun, plural	Americans, Beatles
PDT	Predeterminer	all, both
POS	Possessive ending	's, '
PRP	Personal pronoun	I, you, he
PRP$	Possessive pronoun	my, your, his
RB	Adverb	quickly, very
RBR	Adverb, comparative	faster, better
RBS	Adverb, superlative	fastest, best
RP	Particle	up, off, out

Tag	Meaning	Example
SYM	Symbol	$, %, ©
TO	to (infinitive marker)	to go, to eat
UH	Interjection	wow, oh, oops
VB	Verb, base form	go, eat
VBD	Verb, past tense	went, ate
VBG	Verb, gerund/present participle	going, eating
VBN	Verb, past participle	gone, eaten
VBP	Verb, non-3rd person present	go, eat
VBZ	Verb, 3rd person singular present	goes, eats
WDT	Wh-determiner	which, that
WP	Wh-pronoun	who, what
WP$	Possessive wh-pronoun	whose
WRB	Wh-adverb	where, when, why
.	Sentence-final punctuation	., !, ?
,	Comma	,
:	Colon, semicolon	:, ;
( )	Brackets	(,)
#	Pound sign	#
`` ''	Opening/closing quotes	“ ”

Example : The quick brown fox jumps over the lazy dog.

Word	POS Tag	Meaning
The	DT	Determiner
quick	JJ	Adjective
brown	JJ	Adjective
fox	NN	Noun (Singular)
jumps	VBZ	Verb (3rd person sing. present)
over	IN	Preposition
the	DT	Determiner
lazy	JJ	Adjective
dog	NN	Noun (Singular)

Applications of PoS

(a) Text Preprocessing for NLP Pipelines

- ✓ Tokenization + POS tagging helps in better lemmatization and named entity recognition.
- ✓ E.g., distinguishing between *"flies"* as a noun (insects) and as a verb (action).

(b) Information Retrieval (Search Engines)

- ✓ POS tagging improves query understanding by identifying key terms.
- ✓ Helps filter out stop words and focus on nouns and verbs for better search relevance.

(c) Named Entity Recognition (NER)

- ✓ POS tags help identify names of people, places, organizations.
- ✓ E.g., In *"John works at Google"*, POS helps distinguish "John" (NNP) and "Google" (NNP) as proper nouns.

(d) Machine Translation

- ✓ POS tags help preserve grammatical structure across languages.
- ✓ E.g., helps decide word order while translating *"She is singing"* to other languages like Hindi or French.

(e) Speech Recognition and Text-to-Speech

- ✓ Helps in determining correct intonation and pronunciation.
- ✓ E.g., “lead” (verb) vs “lead” (noun) pronounced differently.

(f) Sentiment Analysis

- ✓ POS tags help identify opinion words (usually adjectives and adverbs).
- ✓ E.g., *“The movie was incredibly good”* — “good” (JJ), “incredibly” (RB) → positive sentiment.

(g) Question Answering Systems

- ✓ Identifies question type based on POS patterns (e.g., "What is..." vs "Who is...").
- ✓ Helps match answers by focusing on the correct noun or verb phrases.

(h) Text Summarization

- ✓ Helps identify main verbs and nouns to extract important information.
- ✓ E.g., Extracting subject-verb-object triplets for summary generation.

(i) Grammar Checking and Correction

- ✓ POS tagging helps detect grammatical errors like wrong verb tense or noun usage.
- ✓ E.g., *"She go to school"* → "go" (VB) is incorrect; should be "goes" (VBZ).

(j) Chatbots and Virtual Assistants

- ✓ Helps bots understand user intent and respond with grammatically correct output.
- ✓ POS tags contribute to intent classification and entity extraction.

2.3 Constituency

- Constituency refers to the hierarchical structure of a sentence where words are grouped into constituents (i.e., sub-phrases or sub-trees), based on syntactic rules.
- It's used in Constituency Parsing, a method of syntactic analysis.
- A constituent is a group of words that function as a single unit within a hierarchical structure.

Goals of Constituency:

- ✓ Understand human language (text or speech)
- ✓ Extract meaning and context
- ✓ Communicate in human-like ways
- ✓ Automate language-based tasks

Constituency Tree

- A constituency tree shows how a sentence is divided into phrases.
- Rules help define constituents:

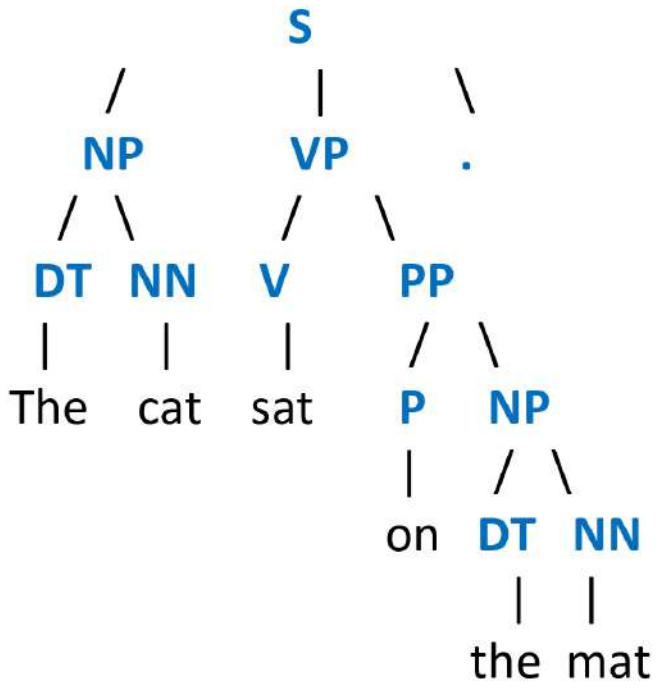
S → NP VP

NP → DT NN

VP → V NP | V PP

PP → P NP

Example : The cat sat on the mat.



S = Sentence

NP = Noun Phrase

VP = Verb Phrase

PP = Prepositional Phrase

DT = Determiner

NN = Noun (singular)

V = Verb

P = Preposition

Applications of Constituency Parsing

- ✓ **Grammar correction** – identifying malformed structures.
- ✓ **Question answering** – extracting phrases as answers.
- ✓ **Machine translation** – preserving grammatical structure.
- ✓ **Text summarization** – extracting key constituents.
- ✓ **Voice interfaces** – identifying sentence meaning via structure.

2.4 Trees

- In NLP, trees are used to represent the syntactic or grammatical structure of sentences.
- The hierarchical structures visually show how words in a sentence relate to each other according to grammar rules.
- A tree in NLP is a graphical representation of the syntactic structure of a sentence.
- Where,

  Nodes = grammatical components (words or phrases)

  Branches = relationships between components

  Root = the top-level syntactic unit (usually the sentence S)

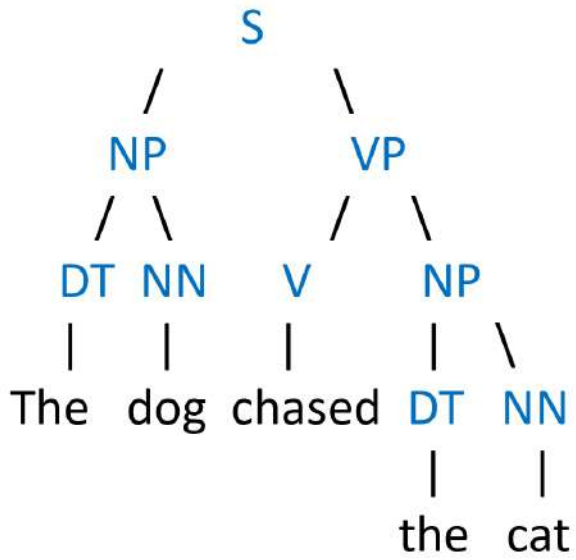
(i) Constituency Tree (Phrase Structure Tree)

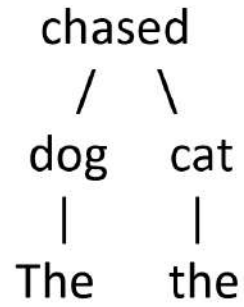
Shows a sentence to broken down into nested phrases (like noun phrases, verb phrases).

(ii) Dependency Tree

Shows grammatical relationships between words (dependencies) with arrows or edges pointing from heads to dependents.

Example : The dog chased the cat.





NP: Noun Phrase → "The dog"

VP: Verb Phrase → "chased the cat"

S: Sentence

"chased" is the main verb

"dog" is the subject (nsubj of "chased")

"cat" is the object (dobj of "chased")

Dependency Tree Vs Constituency Tree

Feature	Constituency Tree	Dependency Tree
Structure	Phrase-based	Word-based (head-dependent)
Grammar model used	CFG (Context-Free Grammar)	Dependency Grammar
Visual output	Tree of nested phrases	Tree with directed links
Tools	NLTK, Stanford Parser	spaCy, Stanza, UDPipe

Applications of Trees

Application	How Trees Help
Syntax Analysis	Understand sentence grammar
Machine Translation	Maintain correct sentence structure in translation
Question Answering	Extract relevant phrases or entities
Text Summarization	Identify main clauses or actions
Information Extraction	Extract subject, verb, object triples
Voice Interfaces	Parse spoken commands grammatically

2.5 Sentence

- A sentence is the basic unit of meaning composed of a sequence of words that conveys a complete thought.
- NLP systems use sentence boundaries to break down and understand text systematically.
- Sentences form the context for understanding grammar and meaning.
- Many NLP tasks operate at the sentence level (e.g., parsing, translation, summarization).
- Identifying sentence boundaries is crucial for tokenization, POS tagging, and semantic analysis.
- In NLP, a sentence is defined by:
  - ✓ Start and end boundaries (often using punctuation like ., ?, !)
  - ✓ Grammatical completeness (subject + predicate)

- NLP tools must handle complexities like:
  - ✓ Abbreviations (Dr., U.S.)
  - ✓ Quotes and brackets
  - ✓ Missing punctuation in informal text (e.g., social media).

Example

Dr. Smith went to the U.S. He arrived on Jan. 5.

Correct Sentences:

Dr. Smith went to the U.S.

He arrived on Jan. 5.

Sentence boundary detection must not split after Dr. or U.S. incorrectly.

Sentence Tokenization (Segmentation)

Tool	Method
NLTK	nltk.sent_tokenize(text)
spaCy	doc.sents after processing
CoreNLP	Advanced sentence splitting with ML

Sentence Structure in NLP

Component	Description	Example
Subject	Who/what the sentence is about	*The cat*
Predicate	What is being said about the subject	*sat on the mat*

Applications of Sentence Processing in NLP

Task	Role of Sentence
POS Tagging	Tags assigned per sentence
Parsing	Tree structures built per sentence
Machine Translation	Translates sentence-by-sentence
Sentiment Analysis	Measures opinion per sentence
Summarization	Extracts key sentences
Chatbots/QA Systems	Understands sentence-level intent

Types of Sentences in NLP

- In NLP, sentences can be categorized in multiple ways based on syntax, semantics, and function.
- Understanding these types helps NLP systems process text accurately for tasks like sentiment analysis, question answering, and summarization.

Category	Sentence Types
Function	Declarative, Interrogative, Imperative, Exclamatory
Structure	Simple, Compound, Complex, Compound-Complex
Sentiment	Positive, Negative, Neutral, Mixed
Dialogue Role	Greeting, Request, Response, Confirmation
Speech Style	Complete, Elliptical, Disfluent

(a) Sentence Function (Speech Acts)

- NLP uses intent classification to detect these types for chatbots and voice assistants.

Sentence Type	Function/Use	Example
Declarative	States a fact or opinion	"The sky is blue."
Interrogative	Asks a question	"What is your name?"
Imperative	Gives a command or request	"Close the door."
Exclamatory	Expresses strong emotion	"Wow! That's amazing!"

(b) Sentence Structure (Syntactic Form)

- Parsing (constituency or dependency) helps identify these structures.

Type	Description	Example
Simple	One independent clause	"He ran."
Compound	Two independent clauses joined by a conjunction	"She came and he left."
Complex	One independent + one or more dependent clauses	"Because it rained, we stayed."
Compound-Complex	Two or more independent + at least one dependent	"We left after he arrived, but it was late."

(c) Sentiment or Polarity (Semantic Category)

- Used in sentiment analysis and opinion mining.

Type	Description	Example
Positive	Expresses a positive feeling	"I love this place."
Negative	Expresses dislike	"This is terrible."
Neutral	Objective or factual	"It is 25 degrees today."
Mixed	Contains both sentiments	"The food was great but slow."

(d) Use in Dialogue (Utterance Role)

- Important in dialogue systems and chatbots.

Type	Role in Conversation	Example
Greeting	Starts interaction	"Hello!"
Request	Asks for something	"Can you help me?"
Response	Answers a query	"Yes, of course."
Confirmation	Confirms or denies info	"That's right." / "No, it's not."

(e) Speech Input (ASR Focus)

- Handled in Automatic Speech Recognition (ASR) and spoken dialogue systems.

Type	Description	Example
Complete	Fully formed with clear intent	“Set a reminder for 5 PM.”
Elliptical	Incomplete or fragmentary	“Tomorrow morning.”
Disfluent	Contains pauses/fillers/errors	“Uhh… I mean… set alarm…”

2.6 Utterance

- In NLP, an utterance refers to a sequence of words spoken or written as a single unit, often representing a complete idea or expression, without concern for grammar or punctuation.
- An utterance is a unit of speech or text produced by a speaker in a given context.
- It can be a sentence, a phrase, or even a single word spoken or written during communication.
- A sentence is grammatically complete; an utterance is context-dependent and can be incomplete but meaningful in a dialogue.
- It is especially relevant in spoken language processing (like chatbots, voice assistants, speech recognition).

Aspect	Details
Utterance	A unit of speech or text produced in context
Types	Declarative, Interrogative, Imperative, Exclamatory, Elliptical, etc.
Importance	Key to understanding conversation, context, user intent
Used In	Chatbots, voice assistants, ASR, QA, dialog systems

Roles of Utterance in NLP:

- ✓ Crucial in dialogue systems and chatbots
- ✓ Needed for speech-to-text and voice assistants
- ✓ Helps model conversational context
- ✓ Useful in intent classification and dialogue act tagging.

Example

Utterance (Text)	Meaning/Intent
"What's the weather?"	Request for weather information
"Turn off the lights"	Command to control a device
"Umm... maybe"	Uncertain confirmation
"I think it's in the box"	Providing information
"No"	Simple rejection

Speaker	Utterance	Notes
A	“What time is it?”	Question
B	“Around 3.”	Incomplete sentence, valid utterance
A	“Thanks.”	Single-word utterance

Key Characteristics:

Feature	Description
Spoken or Typed	Can be spoken (via voice) or written (in chats or messages)
Context-dependent	Often relies on dialogue context or previous utterances
Informal	May not follow strict grammar or sentence rules
No punctuation	Especially in speech, boundaries are not marked by . or ?
May be incomplete	E.g., "Sure." / "Why not?" / "Next one." are valid utterances

Applications of Utterances in NLP

Application	Role of Utterance
Voice Assistants	Each command (e.g., "Play music") is an utterance
Chatbots	Conversations are made of user-bot utterance pairs
Speech Recognition	Transcribes and interprets spoken utterances
Dialogue Systems	Tracks utterance history to manage context
Intent Recognition	Determines what the user wants from an utterance

Types of Utterances in NLP

Type of Utterance	Description	Example
Declarative	States a fact or opinion	"The weather is nice today."
Interrogative	Asks a question	"Are you coming to the class?"
Imperative	Gives a command or request	"Close the door."
Exclamatory	Expresses strong emotion or surprise	"What a wonderful view!"
Elliptical	Incomplete utterance that makes sense in context	"At 3." (in response to a time question)
Backchannel	Short listener responses to show attention	"Uh-huh", "Yeah", "I see"
Acknowledgement	Indicates understanding or acceptance	"Okay", "Got it", "Alright"
Clarification Request	Asks for repetition or more details	"What do you mean?", "Pardon?"
Confirmatory	Repeats or rephrases to confirm understanding	"You said Tuesday, right?"
Greeting/Closing	Used to open or close a conversation	"Hi", "Goodbye", "See you"

Sentence Vs Utterance

Aspect	Sentence	Utterance
Formality	Grammatically correct structure	May be informal or fragmented
Boundaries	Ends with punctuation	Ends when the speaker/writer pauses or stops
Used in	Formal text, documents	Dialogue, speech, chat, voice input
Focus	Syntax and grammar	Speaker intent and context

NLP Tools for Utterance Processing

Tool / Library	Capabilities for Utterance Analysis	Best Used For
NLTK (Natural Language Toolkit)	Tokenization, Part-of-Speech tagging, parsing, dialogue act tagging	Rule-based text analysis of utterances
SpaCy	Tokenization, Named Entity Recognition, intent classification (via add-ons)	Text-based utterance parsing & information extraction
Rasa NLU / Rasa Core	Intent classification, entity recognition, dialogue management	Conversational AI, chatbots
Google Dialogflow	Intent recognition, context tracking, pre-trained utterance models	Voice/text bots, virtual agents
Microsoft LUIS	Intent detection, utterance training, confidence scoring	Enterprise-grade chatbot design
IBM Watson Assistant	Dialogue trees, utterance clustering, slot filling	Business and customer service bots
OpenAI Whisper	Speech-to-text transcription for audio utterances	Converting spoken utterances into text

Contd..

SpeechRecognition (Python Library)	Interface for using speech APIs like Google or Sphinx	Capturing spoken utterances via microphone
DeepSpeech (by Mozilla)	Speech-to-text with deep learning	Real-time utterance capture from audio
AllenNLP	Advanced models for sentence parsing, coreference, semantic role labeling	Research-grade utterance interpretation
Hugging Face Transformers	Pretrained models (BERT, RoBERTa, T5) for classification & context	Classifying and embedding utterances contextually

Utterance Tasks and Tools Mapped

Utterance Task	Recommended Tool(s)
Speech-to-Text (Audio → Text)	OpenAI Whisper, DeepSpeech, SpeechRecognition
Intent Classification	Rasa, Dialogflow, LUIS, HuggingFace Transformers
Dialogue Act Tagging	NLTK, Rasa, AllenNLP
Utterance Embedding	BERT, RoBERTa, Sentence-BERT (HuggingFace)
Entity Recognition in Utterances	SpaCy, Rasa, LUIS, Watson Assistant
Context Tracking in Dialogues	Rasa Core, Dialogflow, IBM Watson
Utterance Tokenization & Parsing	SpaCy, NLTK, AllenNLP

Use Cases for Utterance Tools

Use Case	Tool Example(s)
Voice Assistant (e.g., Alexa, Siri)	Whisper, LUIS, Dialogflow
Chatbot for Customer Support	Rasa, Watson, Dialogflow
Speech Transcription	Whisper, DeepSpeech
Intent and Sentiment Detection	HuggingFace Transformers, SpaCy, Rasa
Training on Custom Utterances	Rasa NLU, LUIS, Dialogflow

2.7 Proposition

- Propositions are language-independent representations of meaning.
- A proposition refers to the underlying meaning or idea expressed by a sentence or utterance — abstracting away from the specific words used.
- A proposition is the core idea or claim conveyed by a sentence — it typically consists of a predicate (action or state) and arguments (entities involved).
- It represents a truth-bearing statement that can be true or false.

Importance of Propositions:

- ✓ Understand meaning beyond syntax
- ✓ Disambiguate between similar sentences with different meanings
- ✓ Perform inference and reasoning
- ✓ Power semantic tasks like:
  - ❖ Question answering
  - ❖ Information extraction
  - ❖ Knowledge graph construction
  - ❖ Paraphrase detection

Tasks with Propositions

Task	What It Does	Propositions Use	Example
Semantic Role Labeling (SRL)	Identifies who did what to whom, when, where, etc.	Extracts the core propositional structure from a sentence	*"Mary gave John a book."* → Give(Mary, Book, John)
Information Extraction (IE)	Pulls facts from unstructured text	Represents facts as subject–predicate–object triples (propositions)	"Einstein was born in Germany.„ → Born(Einstein, Germany)
Open Information Extraction (OpenIE)	Extracts propositions without predefined schema	Breaks down complex sentences into simple, factual propositions	"She wrote and published a book." → Write(She, Book), Publish(She, Book)
Natural Language Inference (NLI) / Textual Entailment	Determines if one sentence implies another	Compares two propositions for entailment, contradiction, or neutrality	Premise: *"A boy is playing football."* Hypothesis: *"A child is playing."*

Contd..

Question Answering (QA)	Answers questions based on documents or knowledge	Matches question propositions with fact propositions	Q: "Who wrote Hamlet?" → Match with Write(Shakespeare, Hamlet)
Semantic Parsing	Converts natural language to formal logical representations	Translates sentences into machine-interpretable propositions	"What is the capital of France?" → Capital(France, ?x)
Summarization	Creates a concise version of the text	Selects core propositions that best represent the content	Original: "He traveled to France and Italy." → Summary: "He traveled to Europe."
Knowledge Graph Construction	Builds structured knowledge from text	Stores facts as triples: subject–relation–object	Paris – is Capital Of – France
Logical Reasoning / Deduction	Derives new truths from known propositions	Uses logical rules on propositions to infer new information	All humans are mortal. + Socrates is human. → Socrates is mortal.
Commonsense Reasoning	Fills in implicit facts	Generates inferred propositions using world knowledge	"He dropped the glass." → The glass broke.

Types of Propositions

Type	Description	Example
Atomic Proposition	Basic unit; expresses a single idea	"The sun is hot." → Hot(Sun)
Compound Proposition	Formed by combining multiple atomic propositions	"It is raining and cold." → Rain ∧ Cold
Negative Proposition	Denies a fact or idea	"The dog is not barking." → ¬Bark(Dog)
Conditional Proposition	If-then relationship between ideas	"If it rains, I'll stay inside." → Rain → Stay Inside
Universal Proposition	Applies to all members of a group	"All cats are mammals." → ∀x (Cat(x) → Mammal(x))
Existential Proposition	States that at least one instance exists	"Some students are failed." → ∃x (Student(x) ∧ Failed(x))
Modal Proposition	Includes necessity, possibility, or belief	"John must be at home." → □ At Home(John)
Temporal Proposition	Includes time-based meaning	"She was singing yesterday." → Past(Sing(She))

Tools & Frameworks Using Propositions

Tool / Library	Capabilities for Propositions	Use Case Example
AllenNLP	Semantic Role Labeling (SRL), textual entailment, coreference	Extract propositions like Who did What to Whom
OpenIE (Stanford)	Open Information Extraction: Subject–Predicate–Object triples	Extracts propositions from open text
PropBank (dataset + API)	Annotated corpus with verbs and their arguments (SRL)	Trains SRL models for proposition structures
FrameNet	Frame semantics for event-structure propositions	Identifies event frames and roles (like buyer, seller)
Semantic Parsing Tools	Maps natural language into logical forms	Parses: "John eats rice" → Eat(John, Rice)
NLTK	Basic parsing, logic tools, inference modules	Represent and evaluate logical forms
spaCy + SRL extensions	Tokenization + third-party SRL models	Can identify verbs, arguments (propositional parts)

Contd..

Hugging Face Transformers	Use of pre-trained models like BART, T5 for entailment & parsing	Converts sentences into structured propositions
AMR Parsers (e.g. Penman)	Abstract Meaning Representation: graph-based proposition capture	Semantic graphs for whole-sentence meanings
COMET / Atomic (Knowledge Models)	Generate inferred commonsense propositions from inputs	"She dropped the glass" → "The glass broke"
TARS (Text-to-Text framework)	Task-agnostic models for entailment and logic tasks	TARS-NLI, TARS-QA

Datasets for Proposition Learning

Dataset	Description	Used By
PropBank	Verb-based semantic roles	AllenNLP, SRL models
FrameNet	Frame semantics and role annotations	Frame-based semantic parsers
SNLI	Sentence pairs with entailment labels	NLI/Entailment models
AMR Bank	Graph-based meaning representations	AMR parsers
Atomic	Commonsense knowledge expressed as propositions	COMET knowledge models

Applications of Proposition

Application	Use of Propositions
Semantic Parsing	Converts natural language to logical/propositional forms
Question Answering	Compares user questions with stored propositions (facts)
Textual Entailment	Checks if one proposition logically follows from another
Knowledge Graphs	Represents propositions as triples: Subject–Predicate–Object
Logical Reasoning (NLI)	Uses propositions to infer, validate or reject arguments
Information Extraction	Extracts subject, verb, and object triples from text
Summarization	Extract core propositions from long text

2.8 Sense

- A sense is a specific meaning or interpretation of a word.
- Sense refers to the specific meaning of a word in a particular context, especially when a word has multiple meanings (polysemy).
- Many words in natural language are ambiguous—they can have multiple meanings (senses) depending on how they are used in a sentence.
- In NLP, understanding the correct sense of a word is crucial for tasks like machine translation, information retrieval, question answering, chatbots and summarization.

Examples of Word Senses with Tools

Word	Sentence	Correct Sense	Tool to Disambiguate
Bank	"She deposited money at the bank."	Financial institution	WordNet, BERT
Bank	"He sat on the bank of the river."	River edge	Lesk Algorithm, WordNet
Plant	"The chemical plant is near the lake."	Industrial facility	Sense2Vec, BabelNet
Plant	"I watered the plant in my room."	Living organism	BERT, WordNet

Types of Senses (Ambiguity)

Type	Description	Example Word	Example Sentences
Polysemy	Word with multiple related meanings	*Paper*	"Read a paper" (article), "Write on paper" (material)
Homonymy	Word with multiple unrelated meanings	*Bat*	"A bat flew" (animal), "Swing a bat" (sports)
Synonymy	Different words with similar meanings	*Begin/Start*	"Begin the game", "Start the match"
Antonymy	Words with opposite meanings	*Hot/Cold*	"The water is hot", "The water is cold"
Hyponymy	Specific instance of a more general word (is-a relationship)	*Rose*	"A rose is a type of flower"
Hypernymy	General word that includes more specific words (parent category)	*Flower*	"Flower" includes rose, lily, daisy, lotus

Types of Senses

Type of Sense	Explanation	Example Sentence	Meaning (Sense)
Lexical Sense	The dictionary meaning of a word in context.	"I deposited cash in the bank."	Financial institution
Contextual Sense	Meaning derived from the surrounding context of the word.	"He sat on the bank of the river."	Edge of a river
WordNet Sense (Synset)	Word grouped in synonym sets in WordNet, each representing one sense.	“Sachin hit the ball to six."	Ball as a sports equipment (WordNet Synset)
Named Entity Sense	Words referring to named entities (person, location, etc.).	"Apple is releasing a new iPhone."	Apple = Company, not fruit
Domain-specific Sense	Meaning relevant only in a specific domain.	"He used a mouse to click the icon."	Mouse = computer device
Idiomatic Sense	Words used as part of an idiom where the meaning is not literal.	"He kicked the bucket last night."	Died (not literal kicking of a bucket)
Metaphorical Sense	Non-literal meaning used figuratively.	"He has a heart of stone."	Emotionless (not actual stone)

Tools and Resources for Word Sense

Tool/Resource	Type	Description
WordNet	Lexical Database	Organizes words into synsets (sets of synonyms), with definitions and examples.
BabelNet	Multilingual Lexical DB	Combines WordNet and Wikipedia for broader coverage across languages.
Lesk Algorithm	Knowledge-based WSD	Disambiguates words using dictionary definitions (overlap-based).
BERT / ELMo	Deep Learning Model	Uses contextual embeddings to dynamically determine sense.
Sense2Vec	Word Embedding	Enhances Word2Vec with part-of-speech tags to distinguish senses.
NLTK	NLP Library (Python)	Provides interfaces for WordNet, Lesk, and other sense-related tools.
SpaCy + WordNet	NLP Pipeline Tool	Can be extended with WordNet to perform WSD.

Impact of Sense Understanding

Application	Impact of Sense Understanding
Machine Translation	Accurate translation depends on correct sense.
Search Engines	Better ranking of relevant results.
Chatbots & Assistants	Proper understanding of user queries.
Text Summarization	Summaries retain the correct meaning.
Question Answering	System picks answers relevant to the right meaning.

2.9 Reference

- Reference is a word or phrase points to something (person, object, etc.).
- A reference is the act of mentioning or alluding to something (a person, place, event, or object) in discourse.
- It refers to how words or phrases (especially pronouns, noun phrases, etc.) point to or identify entities in the real world or within a text.

Uses of Reference:

Task	How Reference Helps
Chatbots/Assistants	To maintain conversational continuity.
Machine Translation	Correct pronoun/gender/number usage across sentences.
Summarization	Maintain clarity by resolving ambiguous references.
Question Answering	Determine exactly who/what the question refers to.
Information Extraction	Link extracted data to the correct entities in text.

Type	Explanation	Example	Reference Points To
Anaphora	Refers backward to something. previously mentioned.	"Sara lost her keys. She looked everywhere."	*She* → Sara
Cataphora	Refers forward to something mentioned later in the sentence or text.	"Before he could speak, Tom started to cry."	*He* → Tom
Exophora (Deixis)	Refers to something outside the text (needs context/world knowledge).	"That is amazing!" (while pointing at a painting)	*That* → Something in the real-world context
Endophora	General term for references within the text (includes anaphora + cataphora).	"Tom met Mike. He smiled."	*He* → Either Tom or Mike (ambiguous case)
Coreference	All expressions that refer to the same entity in a text.	"Lisa told Mary that she would win."	*She* → Lisa or Mary (needs resolution)
Bridging Reference	Refers to something related but not explicitly mentioned.	"I walked into a house. The kitchen was dark."	*Kitchen* → Part of *house* (inferred)
Definite Reference	Assumes the listener/reader already knows the entity.	"The president gave a speech today."	*The president* → Known figure
Indefinite Reference	Refers to something new or unknown to the reader.	"A man walked into the room."	*A man* → Unidentified person

Tools and Techniques for Reference

Tool/Library	Purpose	Features
spaCy	Coreference resolution (via neuralcoref)	Fast, lightweight, supports rule-based methods
AllenNLP	Deep learning-based coreference models	Pretrained coreference models, BERT-based
Stanford CoreNLP	Classical + ML NLP toolkit	Coreference chain extraction, parser
HuggingFace Transformers	Models like BERT for coreference	Used in fine-tuned WSD and co-reference models
NeuralCoref	Add-on to spaCy	For resolving coreferences with neural nets
OntoNotes Dataset	Annotated corpus used for training	Annotated for coreference and sense info

2.10 Referring Expressions

- Referring expression (RE) is a word or phrase used to refer to an entity (person, object, event, etc.) in a discourse or sentence.
- Referring expressions help identify what or who is being talked about.

Examples of Referring Expressions

Sentence	Referring Expression	Refers To
Mary has a dog. She loves it.	She, it	*Mary, dog*
The president gave a speech.	president	The known leader
John met a man. He was nervous.	man, He	Possibly the same person
Give it to that one on the right.	that one	A specific object/person

Types of Referring Expressions

Type	Description	Example
Pronouns	Short words used to avoid repetition	he, she, it, they, this, that, these, those
Definite Descriptions	Assume the listener knows what is referred to	*the man*, *the book on the table*
Indefinite Descriptions	Introduce new entities	*a man*, *some book*
Proper Nouns	Specific names of people, places, things	*John*, *India*, *Microsoft*
Demonstratives	Pointing expressions depending on context	*this*, *that*, *those students*
Zero Anaphora	Reference implied without an actual word (common in some languages)	*(He) Went to school* (subject implied)

Tools and Techniques for Handling Referring Expressions

Tool/Library	Functionality	Notes
CoreNLP (Stanford)	Coreference resolution and mention detection	Can identify referring expressions and their referents
SpaCy	Named Entity Recognition, coref extensions	Supports entity linking and extension for coref
AllenNLP	Deep learning models for coreference resolution	Includes a strong end-to-end neural model
HuggingFace Transformers	Transformers for context understanding	BERT and RoBERTa can be used for coreference tasks
NeuralCoref (with SpaCy)	Coreference resolution model	Adds co-reference functionality to SpaCy
NLTK	Basic parsing and named entity tools	Useful for linguistic analysis

Applications of Referring Expression

Application Area	Use of Referring Expressions
Chatbots/Virtual Assistants	Understanding pronouns and context in ongoing conversation
Question Answering	Resolving references in questions (e.g., *Who is he?*)
Machine Translation	Maintaining correct reference across languages
Summarization	Referring back to earlier parts concisely and clearly
Information Extraction	Identifying entities mentioned in documents
Dialogue Systems	Resolving references to maintain coherent dialogue

2.11 Predicates

- In linguistics and logic, a predicate is the part of a sentence that states something about the subject.
- In NLP, a predicate expresses an action, state, or property of the subject.

Example:

"The cat sat on the mat."

- ✓ Subject: The cat Predicate: sat on the mat

- In logic/semantic representation, a predicate is often viewed as a function:
  - ✓ SAT(cat, mat) → Predicate SAT takes arguments (cat, mat)

Key Concepts

Predicate-Argument Structure:

The predicate and its related entities (subjects, objects).

Semantic Role Labeling (SRL):

Identifying who did what to whom.

First-Order Logic (FOL):

Represents predicates formally with quantifiers and arguments.

Types of Predicates

Type	Description	Example
Verbal Predicate	Uses a verb to express the action/state of the subject	"She runs."
Nominal Predicate	Uses a linking verb + noun/adjective to describe subject	"She is a doctor."
Adjectival Predicate	Describes subject using an adjective	"The flowers are beautiful."
Prepositional Predicate	Starts with a preposition indicating relationship	"The book is on the table."
Compound Predicate	More than one verb/action is linked to the same subject	"He runs and swims every day."
Logical Predicate	Used in logic and AI: function taking arguments	LIKES(John, Mathematics) → John likes Mathematics

Applications of Predicates in NLP & AI

Area	Role of Predicates
Semantic Parsing	Extracting predicate-argument structures from natural language.
Information Extraction	Identifying relationships like works_for(John, Google) from texts.
Question Answering	Mapping questions to predicates and retrieving answer from a knowledge base.
Machine Translation	Maintaining predicate structure across languages to preserve meaning.
Knowledge Graphs	Representing knowledge in triples: (subject, predicate, object)
Logic-Based Reasoning	Used in Prolog/First-order logic to perform inference.
Relation Extraction	Learning to identify predicates that link entities in a sentence.

NLP Tools Supporting Predicates

Tool / Library	Use
spaCy	Dependency parsing, semantic roles (through extensions)
AllenNLP	Semantic Role Labeling (SRL) - extracts predicates and arguments
NLTK	Parsing and basic grammar-based predicate recognition
Stanford CoreNLP	SRL, POS tagging, constituency/dependency parsing
PropBank / VerbNet	Annotated corpora mapping verbs to predicate-argument structures
OpenIE (from Stanford)	Extracts triples (subject, predicate, object) from text
SRL BERT Models	Pretrained models from HuggingFace to extract predicates semantically

2.12 Universe of Discourse

- The Universe of Discourse also known as the domain of discourse.
- UoD is the set of all entities over which variables can range or that can be referred to in a logical or linguistic context.
- UoD is contextual and fundamental to logical reasoning.
- It restricts variables and makes interpretation unambiguous.
- In NLP, UoD is critical for reference resolution, logic, and semantics
- The set of all entities variables can refer to in logic or NLP
- It defines the scope or boundary for reasoning, predicates, quantifiers, and references.
- Determines the scope and meaning of quantified statements and predicates.

Examples:

- **Restricted UoD (e.g., in a classroom):**

  "Everyone passed the exam“

  - ✓ Means: Everyone **in the class**, not in the entire world.

- **In AI Knowledge Base (UoD = all cities in India):**

  - ✓ Predicate: isCapital(x)
  - ✓ Valid values for x: Delhi, Mumbai, Chennai, etc.

Types of Universe of Discourse

Type	Description
Restricted/Finite	Limited to a specific set of known entities.
Open/Infinite	Includes all possible entities in a certain domain (e.g., all real numbers).
Contextual	Defined by the context in which a sentence is used (e.g., a specific document).
Domain-Specific	Based on a particular application like medical, legal, etc.
Temporal UoD	Entities existing or relevant during a specific time frame.
Dynamic UoD	Changes based on the discourse or conversational context.

Applications of Universe of Discourse

Area	Application of UoD
Formal Logic / FOL	Defines domain over which quantifiers like ∀ (for all), ∃ (there exists) apply.
Natural Language Understanding	Helps in resolving references and determining meaning in context.
Semantic Web / Ontologies	Limits the reasoning scope in knowledge representation.
Chatbots and Dialogue Systems	Tracks conversational entities to avoid ambiguity.
Information Retrieval	Filters relevant documents/entities from a specific UoD.
Machine Translation	Ensures accurate translation depending on the domain context.
Expert Systems	Operate within a fixed UoD like legal rules, medical terms, etc.

Universe of Discourse Vs Predicate Logic

Concept	Explanation
∀x P(x)	"For all x in the Universe of Discourse, P(x) is true."
∃x P(x)	"There exists an x in the UoD such that P(x) is true."
Changing UoD	Changes the truth value of logical statements
Logic reasoning	Is valid **only within the current UoD**

2.13 Deixis and Definiteness

(i) Deixis

- Deixis refers to context-dependent words or phrases whose meaning depends on who is speaking, when, and where.
- These expressions are called deictic expressions (or indexicals).
- Meaning is anchored in context.

Examples:

"I", "you", "here", "there", "now", "that", "this", "tomorrow", "yesterday"

Sentence Example:

"I will meet you there tomorrow."

"I" → speaker

"you" → listener

"there" → place

"tomorrow" → time

Types of Deixis

Type	Description	Examples
Person Deixis	Refers to participants in the speech event	I, you, he, she, we
Time Deixis	Refers to time relative to utterance	now, then, today, soon
Place Deixis	Refers to location relative to speaker	here, there, above, near
Discourse Deixis	Refers to parts of the text/discourse	the following, the above
Social Deixis	Reflects social relationships or formality	sir, madam, your honor

(ii) Definiteness

- Definiteness is a semantic feature that determines whether a noun refers to something specific or known.
- Definite expressions refer to something already identified.
- Indefinite expressions introduce new or unknown entities.

Examples:

Definite: *the cat, her car, this book*

Indefinite: *a cat, some person, any book*

Types of Definiteness

Type	Description	Examples
Definite Noun Phrases	Refers to specific known entity	the dog, my house
Indefinite Noun Phrases	Refers to any member of a group/class	a dog, some people
Demonstratives	Specify exact entity and distance	this car, that building
Pronouns	Often definite depending on context	he, she, it
Proper Nouns	Always definite	India, John, Amazon

Applications of Deixis & Definiteness in NLP

Area	Application
Coreference Resolution	Tracking who or what a pronoun or noun refers to (e.g., “he” → “John”)
Dialogue Systems	Understanding user input like “this one” or “over there” in conversational AI
Machine Translation	Preserving correct deixis and definiteness across languages
Information Extraction	Resolving definite/indefinite noun phrases to extract entities/facts
Question Answering	Understanding referential expressions to provide correct answers
Text Summarization	Avoiding ambiguous references in summaries

Tools for Deixis and Definiteness in NLP

Tool/Library	Functionality
spaCy	Named Entity Recognition, POS tagging, Dependency Parsing
AllenNLP	Coreference Resolution, Semantic Role Labeling
Stanford CoreNLP	Pronoun resolution, parsing, and definite reference tracking
NLTK	Part-of-Speech tagging, parsing, basic noun phrase identification
Hugging Face Transformers	Coreference and pronoun resolution with pretrained models (e.g., BERT-based)
NeuralCoref	Add-on for spaCy for resolving coreferences and referents
ELMo / BERT	Contextual embeddings to model deixis in context-sensitive tasks

Example: Coreference & Deixis in Action

Sentence: *"John went to the store. He bought a book."*

"He" is a **person deixis**.

"a book" is **indefinite** (introducing a new entity).

Resolving **"He" → John** is essential for comprehension.

Deixis Vs Definiteness

Aspect	Deixis	Definiteness
Meaning	Refers to context-based expressions	Specifies whether a noun is known/specific
Types	Person, Time, Place, Discourse, Social	Definite, Indefinite, Demonstratives, Pronouns
Applications	Coreference, Dialogue, Translation, QA	Noun resolution, Extraction, Translation
Tools	spaCy, AllenNLP, CoreNLP, NeuralCoref	spaCy, NLTK, HuggingFace

2.14 Lexical Semantics

- Lexical Semantics is a subfield of linguistics and natural language processing (NLP) that deals with the meaning of words, their structure, relationships, and contextual usage.
- Study of word meanings and their relationships
- It answers questions like:
  - ✓ What does a word mean?
  - ✓ How is it related to other words?
  - ✓ How does its meaning change in different contexts?

Example: Word Sense Disambiguation

Sentence 1: "He went to the bank to withdraw money."

Sentence 2: "The boat reached the bank of the river."

Lexical semantics helps determine the correct meaning of "bank" in each sentence using context.

Types of Lexical Semantics

Type	Description	Examples
Polysemy	One word has multiple related meanings	"Bank" (of river, financial institution)
Homonymy	Words sound/spell the same but have unrelated meanings	"Bat" (animal vs. sports equipment)
Synonymy	Words with similar meanings	"Big" ↔ "Large", "Fast" ↔ "Quick"
Antonymy	Words with opposite meanings	"Hot" ↔ "Cold", "Win" ↔ "Lose"
Hyponymy	One word is a subcategory of another	"Rose" is a hyponym of "Flower"
Hypernymy	One word is a broader category than another	"Animal" is a hypernym of "Dog"
Meronymy	Part-whole relationship	"Wheel" is a meronym of "Car"
Holonymy	Whole-part relationship (opposite of meronymy)	"Car" is a holonym of "Wheel"
Lexical Field / Semantic Field	Group of words related by meaning	Colors: red, blue, green
Collocations	Words that frequently occur together	“strong tea”, “make a decision”
Semantic Roles	Role of noun phrases in events (Agent, Theme, etc.)	In “John kicked the ball”, John = Agent

Applications of Lexical Semantics in NLP

Area	Use of Lexical Semantics
Word Sense Disambiguation (WSD)	Determines correct sense of a word in context
Machine Translation	Preserves meaning and correct word usage across languages
Information Retrieval	Improves relevance by understanding synonyms, hypernyms, etc.
Text Summarization	Avoids repetition by grouping similar words (synonyms, hyponyms)
Question Answering	Helps in understanding the exact meaning of words in questions and answers
Chatbots and Virtual Assistants	Better understanding of user queries via word meaning
Semantic Search	Retrieves semantically related documents or phrases
Sentiment Analysis	Recognizes emotional tone even with lexical variations

Tools and Libraries for Lexical Semantics in NLP

Tool / Resource	Functionality
WordNet	Lexical database of English: synonyms, antonyms, hyponyms, hypernyms
NLTK	Interface to WordNet, includes tools for synonyms, antonyms, WSD
spaCy	Tokenization, POS tagging, entity recognition, similarity
Gensim	Word embeddings (Word2Vec, FastText) for capturing semantic similarity
ConceptNet	Common-sense knowledge graph with semantic relationships
BERT / Transformer models	Captures contextual meaning (lexical + compositional)
SemCor / OntoNotes	Annotated corpora for Word Sense Disambiguation and lexical semantics
SentiWordNet	Extension of WordNet that includes sentiment values for words
FrameNet	Database of semantic frames and roles for words
Lexical Semantics APIs	Hugging Face Transformers, AllenNLP for semantic similarity and WSD

Text Book(s):

[1] Coppock, E., & Champollion. L. (2019). Invitation to Formal Semantics. Semantics Boot Camp.

[2] Partee, H. B. (1990). Mathematical methods in Linguistics.

[3] Carnie, A. (2002). Syntax: A Generative Introduction. Malden, M A: Blackwell.

[4] Hurford, R. J., Heasley. B., & Smith B. M. (2007). Semantics: A Coursebook. Cambridge University Press, New York.

[5] Jacobson. P. (2014). Compositional semantics:An interoduction to the Syntax and Semantics Interface. Oxford University Press:NY.

[6] Comorobski. I. (1996). Interrogative Pharases: Syntax and Semantics Interface. Sringer:USA.

Thank You